

Relatório de Auditoria de Segurança

Avaliação Defensiva das 5 Categorias Críticas no Ecossistema GQFit & Fitcoach

DATA DA AUDITORIA

31 de Agosto de 2026

ESCOPO DE CÓDIGO (3 REPOSITÓRIOS)

GuhQuintino/Consultoria, fitcoach, questionarios-acompanhamento

FRAMEWORKS AUDITADOS

Next.js 16/14 (App Router), React 19, Vite, Tailwind CSS

AUDITOR RESPONSÁVEL

Antigravity AI (Security Engineering)

INFRAESTRUTURA DE DADOS

2 Bancos Supabase PostgreSQL (rcebztgx... e nnaadrcm...)

STATUS DA AVALIAÇÃO

Vulnerabilidades Críticas Detectadas

NOTA METODOLÓGICA & ADAPTAÇÃO À STACK

A auditoria foi realizada por inspeção linha a linha do código-fonte real, sem inferências ou suposições. Cada uma das 5 categorias foi mapeada diretamente para as tecnologias em execução: **(1) Banco sem Tranca:** Supabase Row Level Security (RLS) e Stored Procedures SECURITY DEFINER; **(2) Permissão no Navegador:** Gates de UI no React/Next.js versus validação no backend/webhook; **(3) IDOR:** Verificação de posse em identificadores de rotas públicas e autenticadas; **(4) Chaves Expostas:** Verificação de histórico Git, arquivos de configuração e binding no window global; **(5) Inputs sem Tratamento (XSS):** Renderização HTML perigosa (dangerouslySetInnerHTML e innerHTML em event listeners).

Resumo Executivo

A avaliação de segurança cobriu integralmente os 3 repositórios que compõem o ecossistema digital do GQFit (Consultoria Institucional, App Fitcoach Pro e Sistema de Questionários/Dashboard Administrativo). Foram identificados **12 achados acionáveis** e **6 arquiteturas defensivas consolidadas**.

3
CRÍTICA

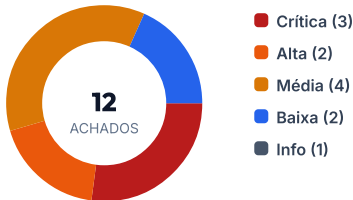
2
ALTA

4
MÉDIA

2
BAIXA

6
PONTOS FORTES

DISTRIBUIÇÃO POR SEVERIDADE



ACHADOS POR CATEGORIA

Banco s/ Tranca	3
Permissão Browser	2
IDOR	2
Chaves Expostas	3
Inputs / XSS	2

Diagnóstico Estrutural

PONTOS FORTES VERIFICADOS

Questionários — RLS Enrijecido: Migração 20260614140000_fix_rls_policies.sql restringe todas as tabelas e storage com verificação estrita de treinador ou service_role.

Questionários — Edge CSRF & Headers: Middleware aplica Content Security Policy estrito, HSTS e validação de cookie double-submit para todas as mutações (POST/PUT/DELETE).

Questionários — Proteção Brute-Force: Bloqueio por IP em 5 tentativas com header Retry-After e respostas genéricas sem vazamento de existência de conta.

Consultoria — Segredo ISR: Rota /api/revalidate exige token coincidente com REVALIDATION_SECRET.

RISCOS CENTRAIS (PONTOS FRACOS)

Fitcoach — Escalada Total de Privilégios: Políticas de RLS em profiles e trigger handle_new_user permitem que qualquer usuário se torne Administrador.

Consultoria — Webhook sem HMAC: Endpoint /api/webhooks/infinitepay processa compras e ativa contas sem validar autenticidade da InfinitePay.

Questionários — Service Key no Git: Chave mestre de serviço (bypassa RLS) commitada no histórico em arquivos de teste legados.

Consultoria — IDOR com Senha em Claro: Rota /api/checkout/status expõe credenciais temporárias por ID de pagamento.

Tabela de Achados Detalhados

SEVERIDADE	ID	ARQUIVO:LINHA	DESCRIÇÃO DA VULNERABILIDADE E IMPACTO
CRÍTICA	FIT-SEC-01	Fitcoach/Full database.sql:191	RLS Permissivo em Profiles (Escalada de Privilégio): A política Users update own profile permite atualizar qualquer coluna da linha própria sem restrição de colunas. Alunos podem alterar role = 'admin' via client SDK.
ALTA	FIT-SEC-02	Fitcoach/Fix Signup Permissions.sql:24	Atribuição Irrestrita de Role no Cadastro: Trigger handle_new_user aceita raw_user_meta_data->>'role' sem filtrar papel 'admin' no registro público.
MÉDIA	FIT-SEC-03	Fitcoach/GetDashboardStats.sql:6	Vazamento em RPC Security Definer: Funções get_coach_dashboard_stats e get_student_gamification_stats não checam se auth.uid() == p_coach_id.
CRÍTICA	CON-SEC-01	Consultoria/webhooks/infinitepay/route.ts:5	Webhook InfinitePay sem Assinatura HMAC: Endpoint cria contas administrativas no Fitcoach e aprova assinaturas sem validar assinatura criptográfica do gateway.
MÉDIA	FIT-SEC-04	Fitcoach/RoleContext.tsx:13	Role Armazenada no LocalStorage: Interface confia em localStorage.getItem('userRole') para roteamento e guardas de acesso na SPA.
ALTA	CON-SEC-02	Consultoria/checkout/status/route.ts:6	IDOR com Senhas Temporárias em Texto Claro: Consulta de status por payment_id público via Service Role retorna fitcoachPasswordTemp e e-mail sem auth.
MÉDIA	QST-SEC-01	Questionários/upload-form/route.ts:30	IDOR no Upload de Fotos de Evolução: Rota pública aceita qualquer alunoId no FormData sem exigir nem validar token de questionário ativo.
CRÍTICA	QST-SEC-02	Questionários/Git:e4c633d (test-db.mjs)	Supabase Service Role Key no Histórico Git: Chave com privilégios de superusuário commitada no repositório; burla 100% das políticas de RLS.
BAIXA	FIT-SEC-05	Fitcoach/lib/supabaseClient.ts:17	Exposição do Cliente no Objeto Global: Atribuição window.supabase = supabase facilita extração de sessão e automação maliciosa via console.
INFO	ALL-SEC-01	Consultoria & QST/.env.local	Tokens e Senhas em Arquivos Locais: Tokens de Telegram, Gemini, Pollinations e senha pooler em texto claro em ambientes locais.
MÉDIA	CON-SEC-03	Consultoria/blog/[slug]/page.tsx:217	HTML de Blog sem Sanitização DOMPurify: Renderização com dangerouslySetInnerHTML de HTML processado via Regex sem uso da biblioteca isomorphic-dompurify.
BAIXA	FIT-SEC-06	Fitcoach/index.html:36	DOM XSS no Global Error Handler: Concatenação direta de e.message em document.body.innerHTML no listener de erro do navegador.

Recomendações Priorizadas

Plano de remediação estruturado por criticidade e janela temporal de mitigação.

[P1] IMEDIATO (24h) — Bloqueio de Vetores Críticos de Acesso

- **Rotacionar a Service Role Key do Supabase:** Gerar nova credencial no painel do Supabase para o banco GQFit (rcebtztxmsywwffeiojbr) e atualizar imediatamente as variáveis de ambiente na Vercel e projetos locais.
- **Corrigir a RLS de profiles no Fitcoach:** Revogar permissão de escrita em role e status para authenticated, permitindo atualização apenas de full_name, phone, avatar_url, preferences.
- **Implementar Validação de HMAC no Webhook InfinitePay:** Bloquear requisições não assinadas no endpoint POST /api/webhooks/infinitepay verificando a assinatura contra INFINITEPAY_WEBHOOK_SECRET.

[P2] ALTA PRIORIDADE (48h) — Proteção de Identidade e Credenciais

- **Remover Senha Temporária da Rota /api/checkout/status:** Excluir fitcoachPasswordTemp do JSON de resposta pública e orientar redefinição por link de primeiro acesso.
- **Travar Papel de Cadastro:** No trigger handle_new_user do Fitcoach, forçar v_role := 'student' por padrão, bloqueando escalada direta via metadados de signup.
- **Validar Token no Upload de Fotos:** Exigir token de questionário na rota POST /api/questionario/upload-foto e validar link ativo antes de aceitar arquivo.

[P3] MÉDIA PRIORIDADE (Próximo Ciclo) — Higiênização e Hardening

- **Sanitarizar Artigos com DOMPurify:** Aplicar DOMPurify.sanitize() no HTML antes de injetar via dangerouslySetInnerHTML em [slug]/page.tsx.
- **Substituir document.body.innerHTML em index.html** por nós de texto ou remover o listener de depuração em build de produção.
- **Adicionar Checagem de Posse em RPCs:** Validar auth.uid() = p_coach_id nas stored procedures estatísticas.

Copie e cole os blocos abaixo diretamente como novas Issues no GitHub para cada repositório correspondente.

--- ISSUE 1 ---

```
**Repositório:** GuhQuintino/fitcoach
**Título:** [Segurança] Escalada de privilégios via RLS permissivo na tabela profiles
**Labels:** security, critical, bug, rls

### Descrição do Problema
A política de Row Level Security (RLS) ``"Users update own profile"`` na tabela ``profiles`` permite que qualquer usuário autentica do atualize o seu próprio registro sem restrição de colunas (``FOR UPDATE USING (auth.uid() = id)``). Como a coluna ``role`` reside nesta mesma tabela, um usuário com papel de aluno (``student``) pode invocar o Supabase SDK no navegador e alterar seu próprio papel para ``admin`` ou ``coach``.

### Evidência
- **Arquivo:** ``Brainstorm/Databases-Implementados/Full database.sql:191``
  ``sql
CREATE POLICY "Users update own profile" ON profiles FOR UPDATE USING (auth.uid() = id);
  ``

### Impacto
Comprometimento total da autorização do banco de dados. Alunos podem obter acesso a rotas administrativas, visões de coach, rotinas de treino de outros alunos e listas completas de usuários.

### Sugestão de Correção
1. Revogar permissão de atualização nas colunas sensíveis (``role``, ``status``, ``id``, ``created_at``) para o papel ``authenticated``:
  ``sql
REVOKE UPDATE ON public.profiles FROM authenticated;
GRANT UPDATE (full_name, avatar_url, phone) ON public.profiles TO authenticated;
  ``

2. Alternativamente, criar um trigger ``BEFORE UPDATE`` que impeça alteração de ``role`` por usuários que não sejam service role.

### Critérios de Aceite
- [ ] Usuário com role ``student`` recebe erro ao tentar executar ``supabase.from('profiles').update({ role: 'admin' }).eq('id', user.id)``.
- [ ] Atualização de dados cadastrais (``full_name``, ``phone``, ``avatar_url``) continua funcionando normalmente.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
**Repositório:** GuhQuintino/Consultoria
**Título:** [Segurança] Ausência de autenticação/HMAC no webhook de pagamentos da InfinitePay
**Labels:** security, critical, payments, webhook

### Descrição do Problema
O endpoint ``POST /api/webhooks/infinitepay`` processa notificações de faturamento e executa ações privilegiadas no banco de dados compartilhado (marcação de pedidos como aprovados) e no Fitcoach Pro (``auth.admin.createUser``, ativação de profiles e extensão de assinaturas em ``students_data``), porém não realiza nenhuma validação de assinatura criptográfica (HMAC) ou token secreto.

### Evidência
- **Arquivo:** ``src/app/api/webhooks/infinitepay/route.ts:5-35``
  ``ts
export async function POST(req: NextRequest) {
  const rawBody = await req.text();
  const payload = JSON.parse(rawBody);
  const orderNsu = payload.order_nsu || '';
  // Nenhuma checagem de assinatura HMAC ou token
}
  ``

### Impacto
Qualquer usuário mal-intencionado pode enviar requisições HTTP forjadas diretamente para a API e provisionar contas gratuitas com prazos de validade estendidos no Fitcoach Pro sem realizar qualquer pagamento real.

### Sugestão de Correção
1. Validar a assinatura enviada no header da InfinitePay contra ``process.env.INFINITEPAY_WEBHOOK_SECRET``.
2. Rejeitar com HTTP 401 qualquer requisição com assinatura ausente ou inválida antes de realizar consultas ao banco.

### Critérios de Aceite
- [ ] Requisições sem header de assinatura retornam ``401 Unauthorized``.
- [ ] Requisições com assinatura forjada retornam ``401 Unauthorized``.
- [ ] Webhooks legítimos assinados pela InfinitePay são processados com sucesso (``200 OK``).
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

****Repositório:**** GuhQuintino/questionarios-acompanhamento
****Título:**** [Segurança] Chave mestra SUPABASE_SERVICE_ROLE_KEY exposta no histórico Git
****Labels:**** security, critical, credentials, leak

Descrição do Problema
A chave de serviço com privilégios administrativos totais (`SUPABASE\_SERVICE\_ROLE\_KEY`) para o banco de dados principal do GQFit (`rcebztgxmsywffeiojbr`) foi commitada no histórico do repositório em arquivos de scripts e testes legados.

Evidência
- ****Commit:**** `e4c633d685bd18c56336e2c476ae313911456e8e`
- ****Arquivo:**** `test-db.mjs:4-5` e `scripts/unificar-alunos.mjs`
 ```.js  
const SUPABASE\_SERVICE\_ROLE\_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';  
```.

Impacto
A `service\_role` do Supabase burla todas as políticas de RLS e concede controle irrestrito de leitura, escrita e exclusão sobre todos os dados pessoais, fotos, fichas de anamnese e transações financeiras.

Sugestão de Correção
1. Acessar o Dashboard do Supabase (Project Settings → API) e gerar uma nova `service\_role secret key`.
2. Atualizar as variáveis de ambiente na Vercel e nos arquivos `.env.local` de desenvolvimento.
3. Utilizar ferramentas como `git-filter-repo` ou `BFG Repo-Cleaner` para purgar o segredo do histórico do GitHub.

Critérios de Aceite
- [] A chave antiga foi revogada no painel do Supabase.
- [] A nova chave foi configurada nas variáveis de produção.
- [] Todos os serviços continuam operando normalmente com a nova credencial.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

****Repositório:**** GuhQuintino/Consultoria
****Título:**** [Segurança] IDOR expõe senhas temporárias em texto claro em `/api/checkout/status`
****Labels:**** security, high, idor, auth

Descrição do Problema
O endpoint `GET /api/checkout/status?payment\_id=<id>` consulta a tabela `payments` utilizando o `createServiceClient()` (que ignora RLS) e retorna o e-mail do aluno e a senha temporária (`fitcoachPasswordTemp`) gerada durante o fluxo de compra, sem autenticação de sessão ou validação de posse.

Evidência
- ****Arquivo:**** `src/app/api/checkout/status/route.ts:28-34`
 ```.ts  
return NextResponse.json({  
 status: payment.status,  
 provisioningStatus: payment.provisioning\_status,  
 planId: payment.plan\_id || null,  
 fitcoachEmail: payment.metadata?.fitcoach\_email || null,  
 fitcoachPasswordTemp: payment.metadata?.fitcoach\_password\_temp || null,  
});

**### Impacto**  
Qualquer terceiro que intercepte ou adivinhe um `payment\_id` pode visualizar as credenciais de primeiro acesso do aluno antes mesmo dele realizar o primeiro login.

**### Sugestão de Correção**  
1. Remover o campo `fitcoachPasswordTemp` da resposta da API de status.  
2. Instruir o aluno a definir sua senha através do link de primeiro acesso ou enviar as instruções por canal seguro (e-mail transacional autenticado ou tela pós-checkout protegida por token de sessão efêmero).

**### Critérios de Aceite**  
- [ ] O endpoint `/api/checkout/status` retorna apenas status da transação (`status`, `provisioningStatus`), sem credenciais.

### --- FIM ISSUE 4 ---

#### --- ISSUE 5 ---

**\*\*Repositório:\*\*** GuhQuintino/questionarios-acompanhamento

**\*\*Título:\*\*** [Segurança] IDOR no endpoint de upload de fotos de evolução por ausência de validação de token

**\*\*Labels:\*\*** security, medium, idor, storage

##### ### Descrição do Problema

O endpoint `POST /api/questionario/upload-foto` é público no middleware, mas aceita diretamente o campo `alunoId` vindo do Form Data sem validar se existe um token de questionário ativo (`link\_questionario`) associado àquele aluno.

##### ### Evidência

```
- **Arquivo:** `src/app/api/questionario/upload-foto/route.ts:43-52, 98-102`
...
ts
const alunoId = formData.get('alunoId');
const storagePath = await fotoService.uploadFotoStorage(alunoId.trim(), buffer, file.type);
...
```

##### ### Impacto

Um atacante que possua o identificador UUID de um aluno pode realizar uploads não solicitados e poluir o bucket privado de fotos de evolução daquele aluno, consumindo armazenamento e interferindo no histórico de avaliação.

##### ### Sugestão de Correção

1. Exigir o campo `token` no FormData de upload.
2. Resolver e validar o link através de `LinkService.validarLink(token)` antes de autorizar o envio ao Supabase Storage.

##### ### Critérios de Aceite

- [ ] Requisições para `/api/questionario/upload-foto` sem `token` válido ou com token expirado/respondido são rejeitadas com status 400/403.

#### --- FIM ISSUE 5 ---

#### --- ISSUE 6 ---

**\*\*Repositório:\*\*** GuhQuintino/Consultoria & fitcoach

**\*\*Título:\*\*** [Segurança] Sanitização de HTML com DOMPurify em artigos de blog e correção de DOM XSS no error handler

**\*\*Labels:\*\*** security, medium, xss, frontend

##### ### Descrição do Problema

1. Em **\*\*Consultoria\*\*** (`src/app/blog/[slug]/page.tsx:217`), o conteúdo do artigo é transformado por regex e injetado via `dangerouslySetInnerHTML` sem passar por `DOMPurify`, permitindo execução de XSS caso scripts maliciosos residam no banco.
2. Em **\*\*Fitcoach\*\*** (`index.html:36`), o listener global de erros concatena `e.message` diretamente em `document.body.innerHTML`, gerando risco de DOM XSS.

##### ### Evidência

```
- **Consultoria:** `src/app/blog/[slug]/page.tsx:217`
...
tsx
<div dangerouslySetInnerHTML={{ __html: otimizarHtmlDoArtigo(post.content, post.title) }} />
...
- **Fitcoach:** `index.html:36`
...
js
window.addEventListener('error', function (e) {
 document.body.innerHTML += '<div>... ' + e.message + '</div>';
});
...
```

##### ### Impacto

Possibilidade de execução de scripts arbitrários no contexto de segurança dos visitantes do blog ou usuários do app Fitcoach.

##### ### Sugestão de Correção

1. Em `[slug]/page.tsx`, importar `DOMPurify` de `isomorphic-dompurify` e sanitizar o retorno de `otimizarHtmlDoArtigo` antes de renderizar.
2. Em `index.html`, remover a manipulação de `document.body.innerHTML` no listener de erro ou sanitizar a mensagem de erro com `textContent`.

##### ### Critérios de Aceite

- [ ] Tags `